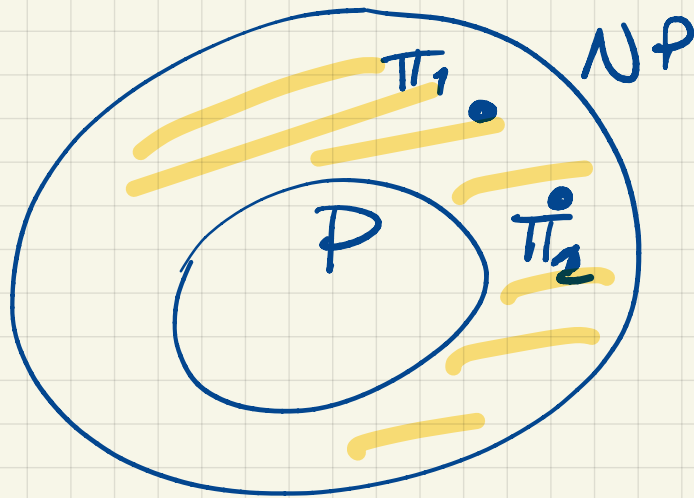


Def. problemi di decisione



P, NP



$\exists$ problemi che sono in NP
e se qualcuno riuscisse a mostrare
nessuno $\bar{e}$ riuscito a mostrare
che sono in P (non $\exists$
un algoritmo polinomiale che li
risolve!)

OBBIETTIVO: definire quando
un problema "non $\bar{e}$ in P"
e meno che $P = NP$

SAT (satisficibilità di una formula CNF) è il primo problema che non è in P e mai che $P = NP$!

$SAT \in NP$ ($\exists$ un algoritmo polinomiale verificatore!)

■ viene dimostrato facendo vedere che ogni problema in NP è RISOLTO USANDO SAT!

■ ne consegue che se $SAT \in P$, allora risolve ogni problema in NP con una chiamata a SAT che è polinomiale cioè $NP = P$

■ quindi se $NP \neq P$
SAT non può essere in P !

TEOREMA di Cook

SAT è NP-completo.

Def. di problema π NP-completo

INTUITIVAMENTE:

(1) π è in NP

(2) $\forall$ problema $\pi' \in NP$,
esiste un modo di risolvere
 π' USANDO π .

• RIDUZIONE POLINOMIALE TRA
problemi di decisione

(1) funzione f calcolabile in Tempo
polinomiale che prende in

input una istanza x del problema A e lo trasforma in un input per il problema B , cioè $f(x) = \text{input per il problema } B$.

(2) vale la seguente proprietà

A su input x restituisce YES
sse

B su input $f(x)$ restituisce YES

Riduzione polinomiale da A a B
e si scrive in sintesi che

$A \leq_p B$ se A si RIDUCE

a B (cioè $\exists$ riduzione polinomiale da A a B)

Def. Π (problema) NP-completo

(1) $\Pi \in \text{NP}$ ($a \in \text{Casse}$)

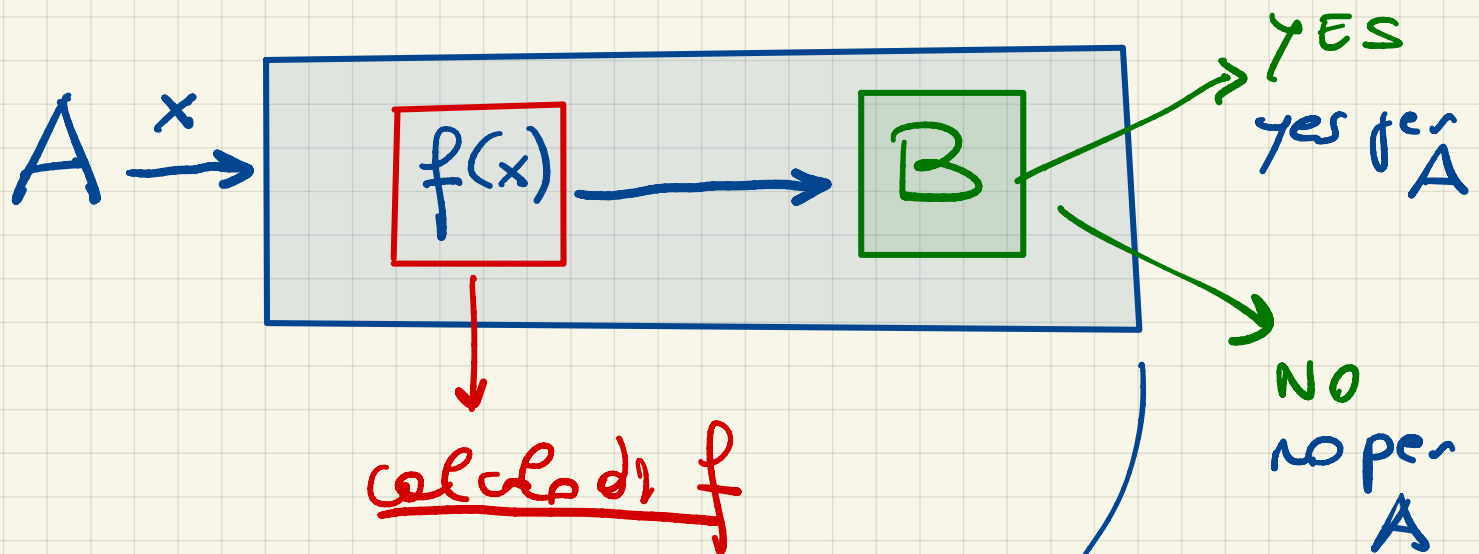
(2) $\forall \Pi' \in \text{NP}$, $\exists$ una riduzione polinomiale da Π' a Π ,
cioè

$$\Pi' \leq_p \Pi !$$

esempi di problemi NP-completi
OLTRE a SAT da per il
Teorema di Cook è NP-completo!

- CLIQUE
- VERTEX-COVER
- HAMILTONIAN CYCLE ...
- TSP-d

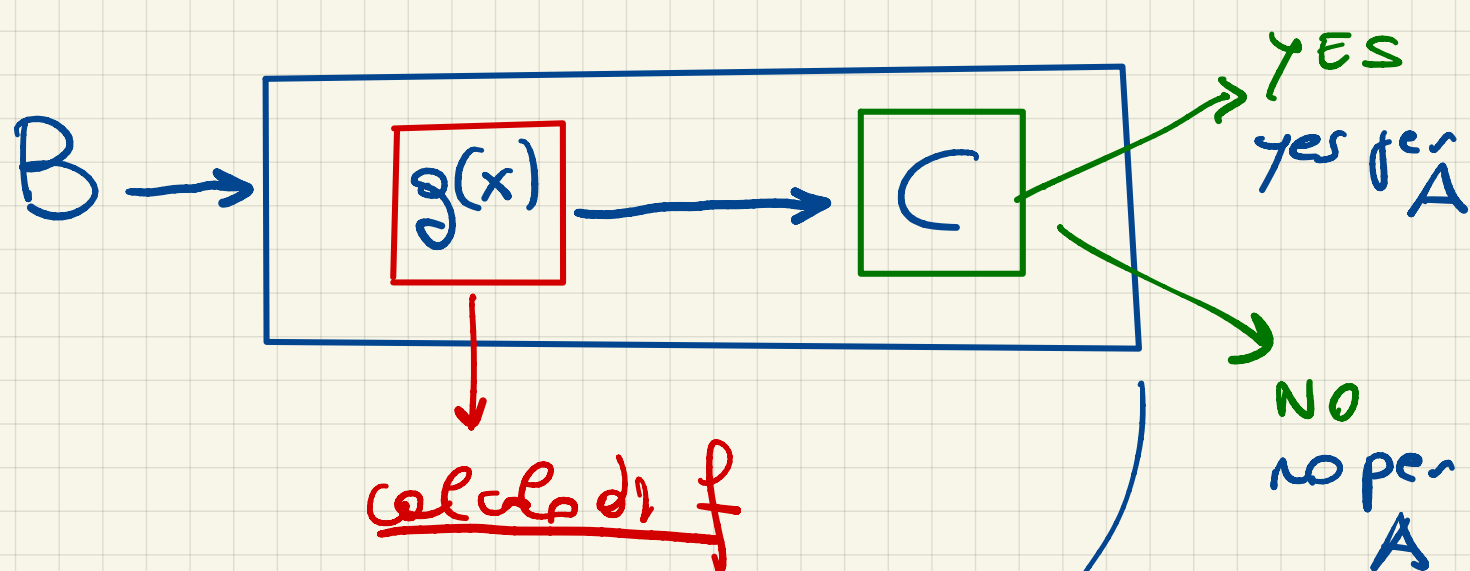
Def. ALTERNATIVA di
 problema NP-completo che
 si può usare a "lavoro polico".



l'algoritmo che risolve A

$A <_p B$ (scheme delle
 riduzioni)

se $B <_p C$ deduco
 $\Rightarrow$



$$B \leq_p C$$



se $A \leq_p B$ e $B \leq_p C$
 dove la prima riduzione è la
 funzione f e la seconda
 riduzione è la funzione g
 allora

$g \circ f$ è una

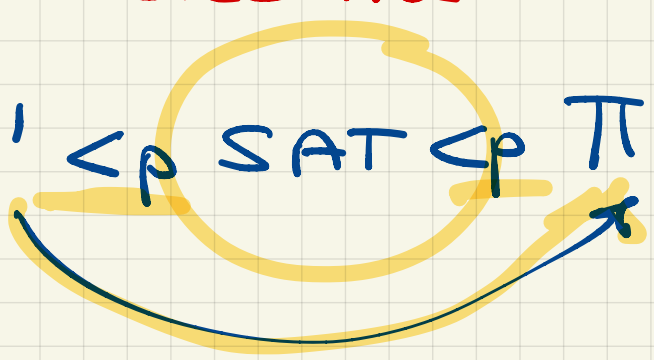
riduzione da A a C
 quindi $\leq_p$ è transitiva!

$$\forall \pi' \in NP, \quad \pi' <_P SAT$$

ho un nuovo problema
 di decisione π''
 NP-difficile (cioè (2))
 $\forall \pi' \in NP, \quad \pi' <_P \pi''$

• $\forall \pi' \in NP, \quad \pi' <_P \pi''$
 per dimostrare che π'' è NP-difficile
 (2)

$SAT <_P \pi''$ allora
 per transitività siccome

$$\forall \pi' \in NP, \quad \pi' <_P SAT <_P \pi''$$


deduco che $\forall \pi', \quad \pi' <_P \pi''$

cioè Π'' è NP-difficile

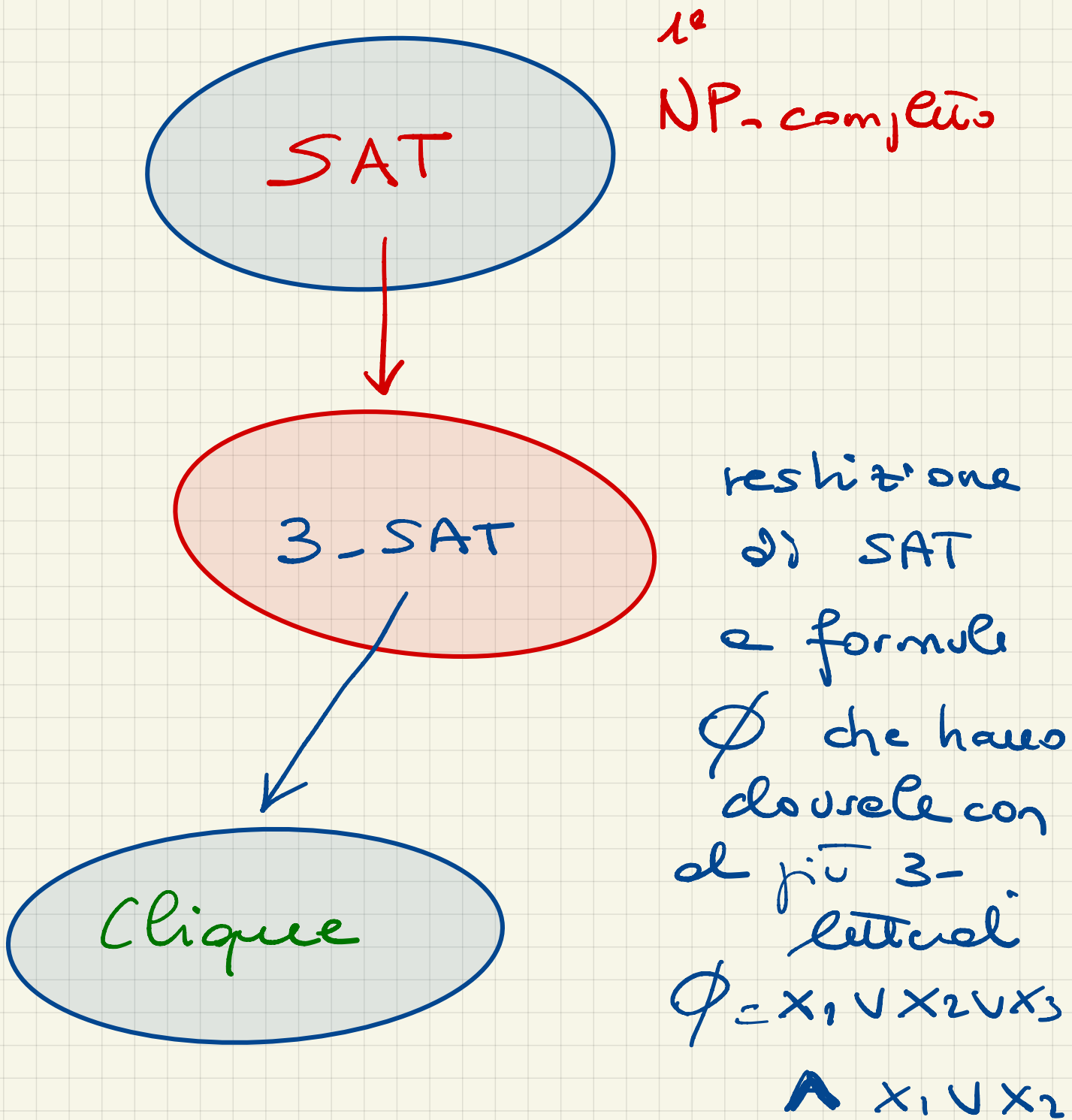
$\Downarrow$ per transitività della riduzione polinomiale un qualunque problema NP-completo può essere usato per dimostrare che un NUOVO problema è NP-completo.

Def. che usiamo in pratica per dimostrare che un problema A è NP-completo è la seguente:

(1) $A \in NP$

(2) $\exists B$, B NP-completo e $B \leq_p A$.

$\Downarrow$ A è almeno difficile
quanto B



Teorema

Clique $\bar{c}$ NP-completo

Problema Clique

INPUT: $G = (V, E)$, un
parametro K

OUTPUT: YES/NO esiste

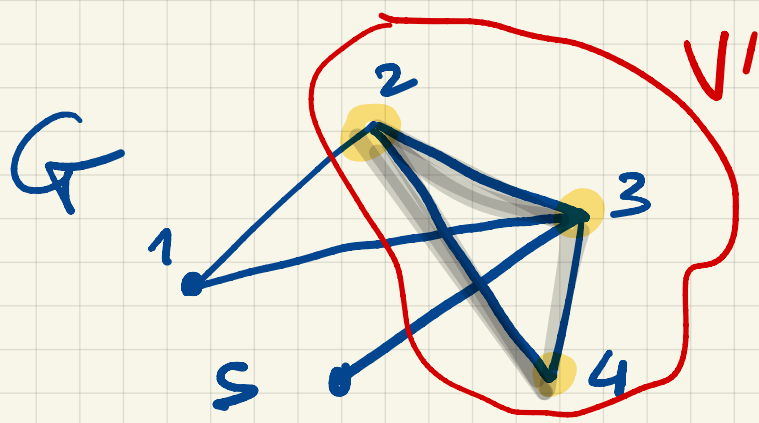
una clique in G di
 K vertici $\implies$ clique

V' è clique

$V' \subseteq V$ tale che

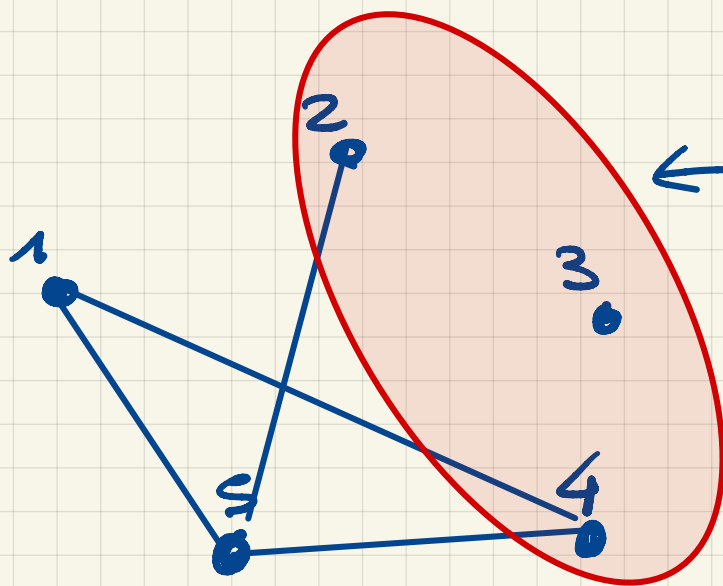
$\forall (u, v) \in V' \times V' - \{(v, v) : v \in V\}$

$(u, v) \in E$



$K=3$ c'è
la clique
in G $\{2, 3, 4\}$

$G \longrightarrow \overline{G}$ (che è
 il grafo che ha i non
 "archi" di G



insieme
 indipendente
 cioè
 non ci
 sono archi
 di collegamento

se prendo il complemento

$\overline{G}$ l'insieme di vertici

$V - V'$ (dove V' è clique)

è un VERTEX-COVER!

Ridurre 3-SAT a clique

Essay

$$\begin{cases} C_1 = x_1 \vee \neg x_2 \vee \neg x_3 \\ C_2 = \neg x_1 \vee x_2 \vee x_3 \\ C_3 = x_1 \vee x_2 \vee x_3 \end{cases}$$

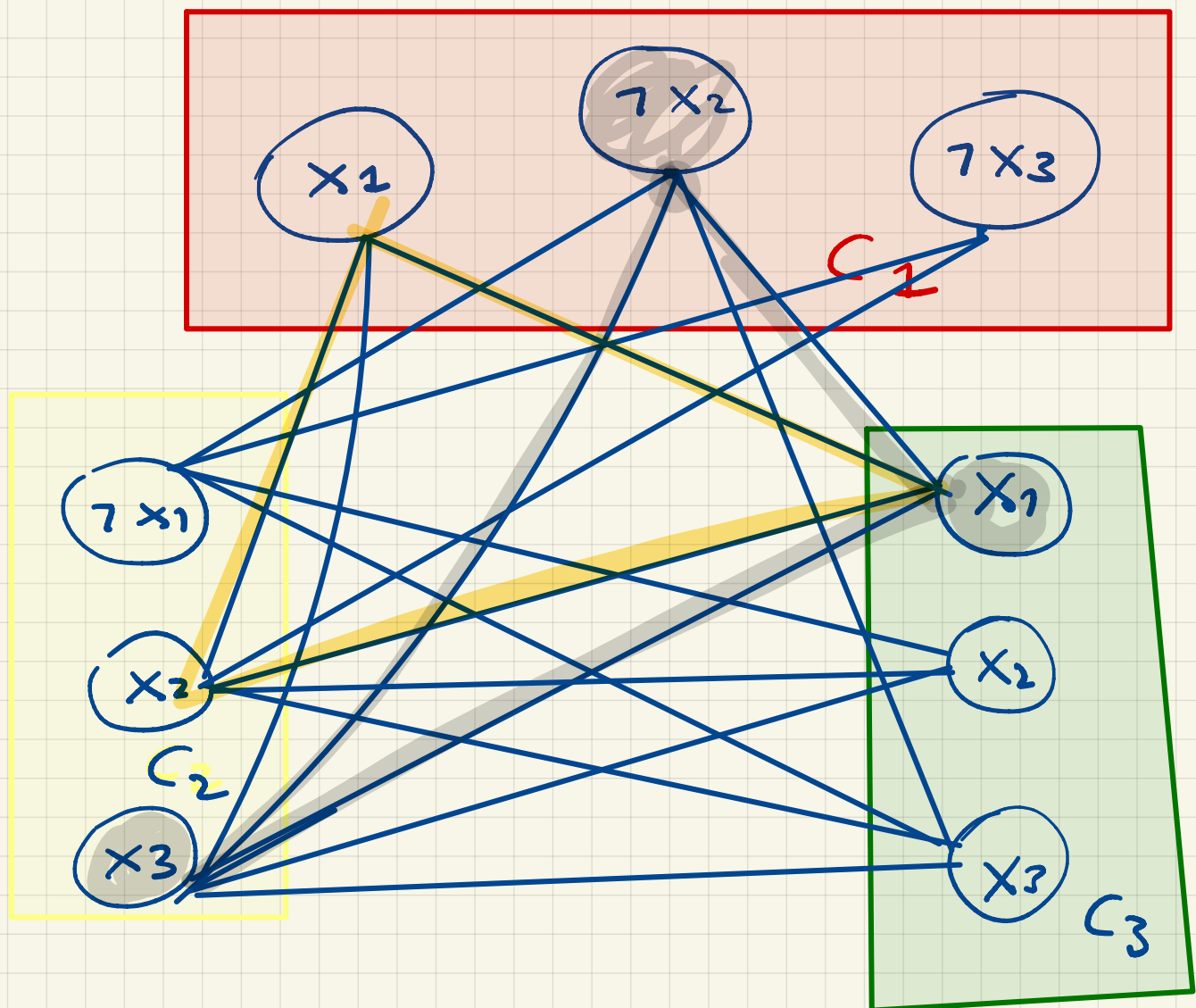
trasformo ogni clausola in
un "gadget" che è un grafo
 $\Rightarrow$ poi metto assieme
i "gadget"

IDEA: traduco ogni clausola
in un grafo tale per cui nella
clique prendo 1 e 1 solo
VERTICE di tale grafo -

cioè se ϕ ha K

clausole allora il grafo
avrà una clique di
dim. $K \rightarrow$

$$\begin{cases} C_1 = x_1 \vee \neg x_2 \vee \neg x_3 \\ C_2 = \neg x_1 \vee x_2 \vee x_3 \\ C_3 = x_1 \vee x_2 \vee x_3 \end{cases}$$



$$\left\{ \begin{array}{llll} \neg x_2 & \overline{e} & \text{true} & x_2 = 0 \\ \cdot x_3 & \overline{e} & \text{true} & x_3 = 1 \\ \cdot x_1 & \overline{e} & \text{true} & x_1 = 1 \end{array} \right\} \neg x_2, x_3, x_1 \}$$

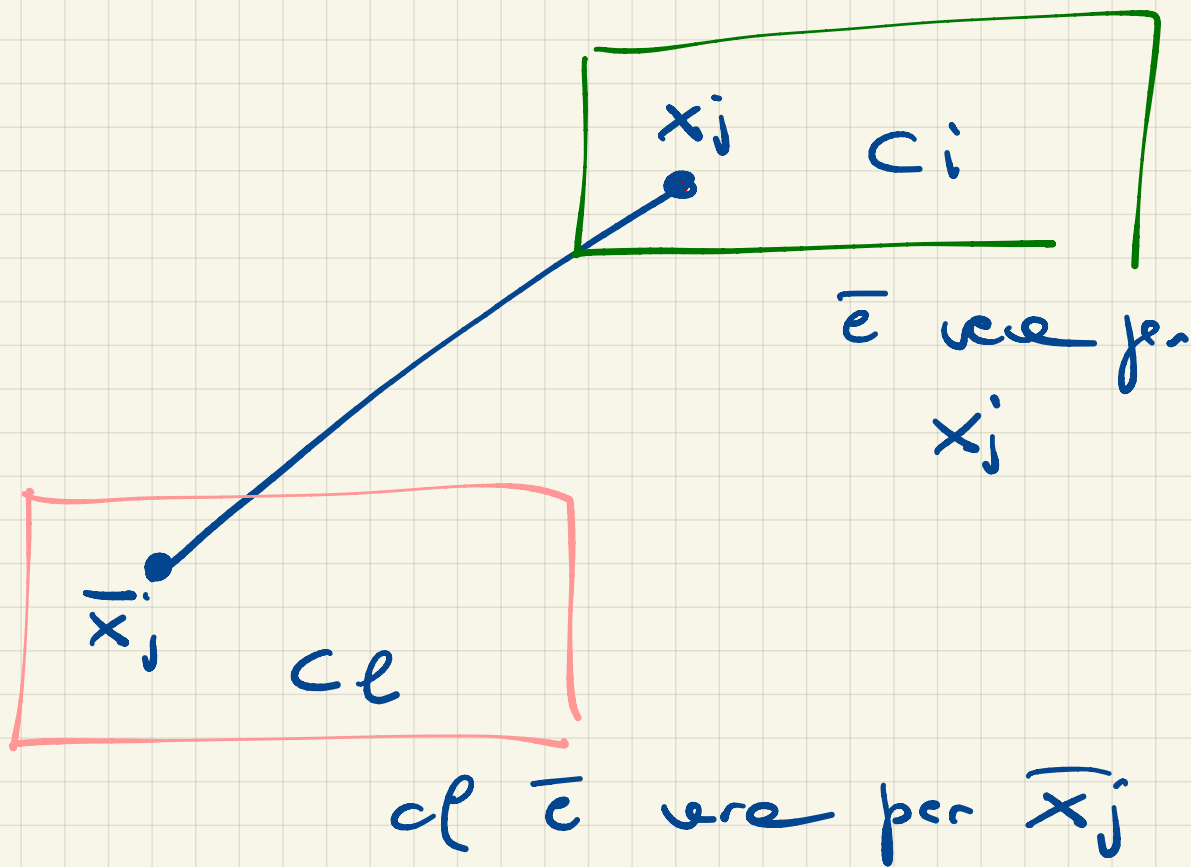
clique

$$\begin{array}{l} x_1 = 1 \\ x_2 = 1 \end{array}$$

$$\{x_1, x_1, x_2\}$$

- Non connessi i vertici delle singole clausole e in questo modo prende 1 e un solo vertice per ogni clausola.
- due codificano il fatto che se ci è vero per un letterale x_j ($\bar{x}_j$) allora non posso rendere vera un'altra clausola c'è con la negazione di x_j (cioè $\bar{x}_j$) (x_j)

Non posso avere che nella clique del prefisso sia presente sia x_j che $\bar{x}_j$.



Riepilogo

Complete la riduzione

• $3\text{SAT} \leq_p \text{CLIQUE}$

• $3\text{SAT} \leq_p \text{IND-SET}$

→ $\text{CLIQUE} \leq_p \text{VERTEX-COVER}$